

Nocturne Archive

A self-correcting Retrieval-Augmented Generation assistant that answers questions over your own documents — and verifies its own answers before trusting them.

Built by **Divyanshu** · Python · Local embeddings · ChromaDB · Groq / Gemini LLM · 100% free stack

1. What is this project?

Nocturne Archive is a **RAG (Retrieval-Augmented Generation)** knowledge assistant. You upload your own documents (PDF, DOCX, TXT, MD); the system reads and indexes them, and then answers your questions **strictly from the content of those documents** — with a source citation, a confidence score, and an honest "I don't know" when the answer isn't in your files.

Unlike most "chat with PDF" demos that stop at *retrieve* → *answer*, this project adds a **multi-agent self-correction loop**: after drafting an answer, a second "critic" agent fact-checks it against the retrieved text. If confidence is low, the query is automatically rewritten and retrieval is retried. This is what meaningfully reduces hallucination.

The core idea in one line: retrieve the most relevant passages from your documents, generate an answer grounded only in them, then **verify that answer against the evidence** — and refine if it doesn't hold up.

RAG · Multi-agent · Self-correction · Hybrid retrieval · Cross-encoder re-rank · Citations · Confidence scoring · Streaming · Conversation memory · Evaluation harness · Tested + CI

2. What makes it different

Feature	Why it matters
Self-correction loop (multi-agent)	An <i>answer agent</i> drafts the reply, a <i>critic agent</i> fact-checks it against the retrieved context, and if confidence is low the query is rewritten and retrieval retried — cutting hallucination.
Structured ingestion	PyMuPDF extracts text <i>and</i> detects tables, converting them to Markdown so rows/columns survive chunking — plus accurate page numbers for citations.
Hybrid retrieval + re-ranker	Semantic (BGE embeddings) + BM25 keyword search fetch a wide candidate pool; a cross-encoder then re-reads each (query, passage) pair to rank precisely.
Confidence + citations + "I don't know"	Every answer ships with a confidence score, the exact source file + page, and refuses to guess when the documents don't contain the answer.
Evidence inspection	Click any citation to reveal the exact passage that grounds the claim — full traceability.
Conversation memory	Follow-up questions are condensed into standalone queries using chat history, so "and what about sick leave?" just works.
Token streaming	Answers stream token-by-token (SSE), then settle into the verified result.
Evaluation harness + CI	A golden Q&A set measures answer correctness, retrieval hit-rate and "I don't know" honesty; a mocked <code>pytest</code> suite runs offline on every push.

3. Technology stack

Language & core

- **Python 3.10+** — entire backend
- **FastAPI + Uvicorn** — web server / API
- **python-dotenv** — config via `.env`

Ingestion

- **PyMuPDF (fitz)** — PDF text + table extraction
- **pypdf** — fallback PDF reader
- **python-docx** — DOCX files
- Native TXT / Markdown loaders

Embeddings & retrieval

- **sentence-transformers** (local, free)
- **BGE** `bge-small-en-v1.5` — 384-dim embeddings
- **rank-bm25** — keyword search
- **Cross-encoder** `ms-marco-MiniLM-L-6-v2` re-ranker

Vector store

- **ChromaDB** — persistent vector DB
- Cosine similarity over normalized vectors
- Per-document add / delete management

LLM (pluggable)

- **Groq** — `llama-3.3-70b-versatile` (default, fast free tier)
- **Google Gemini** — `gemini-2.5-flash-lite`
- Streaming + JSON output + rate-limit retry

Frontend & quality

- Custom **HTML / CSS / vanilla JS** UI (this "Nocturne Archive")
- Optional **Streamlit** UI + **CLI**
- **pytest** suite (mocked, offline) + **GitHub Actions** CI

Design principle — everything free & local where possible: embeddings and re-ranking run on-device via sentence-transformers (no API cost, no rate limits); only the final answer/critic generation calls a free-tier LLM. No Docker required.

4. How it works — the two flows

Flow A · Indexing (when you upload documents)

- 1 **Ingest** — `ingest.py` loads each file. For PDFs, PyMuPDF extracts per-page text *and* detects tables, converting them to Markdown. Every piece keeps its `source` filename and `page` number as metadata.
- 2 **Chunk** — `chunker.py` splits text into ~800-character overlapping chunks along paragraph boundaries, so an idea is never cut in half. A ~120-char overlap preserves continuity between chunks.
- 3 **Embed** — `embedder.py` turns each chunk into a 384-dim vector using the local BGE model (meaning → numbers). Vectors are normalized for cosine similarity.

- 4 **Store** — `vectorstore.py` saves vectors + text + metadata into **ChromaDB**, persisted to disk so the index survives restarts.

Flow B · Answering (when you ask a question)

- 1 **Condense (memory)** — if it's a follow-up, `agents.py` rewrites it into a standalone question using recent chat history (resolves "it" / "that").
- 2 **Retrieve (hybrid)** — `retriever.py` runs semantic vector search *and* BM25 keyword search over a wide candidate pool (~20), then fuses the two scores (weighted 60% semantic / 40% keyword).
- 3 **Re-rank** — `reranker.py` runs a cross-encoder that reads the (query, passage) pairs together and re-scores them precisely, keeping the top 5.
- 4 **Answer** — the **answer agent** (LLM) writes an answer using **only** the retrieved context, with bracket citations `[1] [2]`, streaming token-by-token.
- 5 **Critique** — the **critic agent** (LLM) fact-checks the answer against the same context and returns JSON: `grounded?`, `confidence`, and what's `missing`.
- 6 **Refine (self-correct)** — if confidence is below threshold, the query is rewritten from what was "missing" and steps 2–5 repeat (up to a few tries). The best-scoring attempt wins.
- 7 **Deliver** — final answer + confidence + citations + the exact evidence passages are returned to the UI.

5. The self-correction loop (the differentiator)

This is the piece most beginner RAG projects skip. Three cooperating LLM roles run in a loop defined in `agents.py` :

- **Answer agent** — produces a cited answer strictly from the context; is instructed to reply *"I don't know based on the provided documents"* when the context lacks the answer.
- **Critic agent** — a strict fact-checker that returns structured JSON (`grounded` , `confidence 0-1` , `reason` , `missing`). An honest "I don't know" is correctly scored as grounded.
- **Query rewriter** — when the critic isn't satisfied, it turns the "what's missing" hint into an improved search query (adds synonyms / key terms) and retrieval is retried.

Loop: retrieve → answer → critique → (*if weak*) rewrite + re-retrieve → answer ... up to `MAX_REFINE_TRIES` .
The attempt with the highest confidence is returned, and the answer is flagged `self_corrected` if more than one pass ran.

6. Project structure

Path	Responsibility
<code>server.py</code>	FastAPI backend serving the web UI + JSON API (upload, ask-stream, documents, stats).
<code>web/</code>	Custom animated frontend — <code>index.html</code> , <code>styles.css</code> , <code>app.js</code> .
<code>app.py</code>	Alternative Streamlit UI (zero frontend code).
<code>cli.py</code>	Command-line interface (index / ask / stats).
<code>src/config.py</code>	All tunable settings (chunk size, models, top-k, thresholds).
<code>src/ingest.py</code>	Load PDF/DOCX/TXT/MD → text + metadata.
<code>src/chunker.py</code>	Overlapping, paragraph-aware chunking.
<code>src/embedder.py</code>	Local sentence-transformers embeddings.
<code>src/vectorstore.py</code>	ChromaDB store + per-document management.
<code>src/retriever.py</code>	Hybrid retrieval (semantic + BM25 fusion).
<code>src/reranker.py</code>	Cross-encoder re-ranking.
<code>src/llm.py</code>	Pluggable Groq/Gemini wrapper (+ streaming, JSON, retry).
<code>src/agents.py</code>	Answer + critic + self-correction + conversation memory.
<code>src/pipeline.py</code>	High-level <code>build_index()</code> / <code>answer()</code> orchestration.
<code>eval/</code>	Golden Q&A dataset + metrics harness.
<code>tests/</code>	pytest suite (mocked, offline) — gated by GitHub Actions CI.

7. Running the project

Install & configure

1. `pip install -r requirements.txt`
2. Copy `.env.example` → `.env` and paste one free key (**Groq** recommended, or **Gemini**).

Run (any one)

- **Web UI:** `python -m uvicorn server:app --port 8000` → open `http://localhost:8000`
- **Streamlit UI:** `streamlit run app.py`
- **CLI:** `python cli.py index` then `python cli.py ask "..."`

Note on first run: the embedding and re-ranker models (~130 MB) download once on the first query, so the first answer is slow; every answer after that is fast. This is model loading — not the LLM API.

8. Key configuration knobs

All in `src/config.py` (overridable via `.env`):

Setting	Default	Meaning
<code>CHUNK_SIZE / OVERLAP</code>	800 / 120	Chunk length and overlap in characters.
<code>TOP_K</code>	5	Passages fed to the LLM after re-ranking.
<code>RETRIEVE_CANDIDATES</code>	20	Wide pool fetched before re-ranking.
<code>HYBRID_ALPHA</code>	0.6	Weight of semantic vs. BM25 in fusion.
<code>CONFIDENCE_THRESHOLD</code>	0.6	Below this, the self-correction loop retries.
<code>MAX_REFINE_TRIES</code>	2	Extra retrieval passes allowed.
<code>HISTORY_TURNS</code>	4	Prior turns used to condense a follow-up.